

Developing Custom Executors

One of AttackMate's design goals is extensibility. If you need a command type that does not exist yet (wrapping a proprietary tool, talking to an internal API, or implementing domain-specific logic) you can add it by following a straightforward pattern.

Architecture Recap

When AttackMate runs a playbook, each command goes through this chain:

```
YAML command
  |
  ▼
Command schema (Pydantic)
  | validates fields, sets defaults
  ▼
ExecutorFactory
  | looks up the right executor by `type`
  ▼
Executor._exec_cmd()
  | does the actual work
  ▼
Result(stdout, returncode)
```

Your job when adding a new command is to:

1. Define the **schema** (what fields the YAML command accepts)
2. Implement the **executor** (what happens when the command runs)
3. Wire everything together (register, import, update union types)

Step-by-Step Guide

Step 1: Define the Schema

Create a file in `src/attackmate/schemas/`. Inherit from `BaseCommand` and register it with the `CommandRegistry`:

```
# src/attackmate/schemas/hello.py
from typing import Literal
from .base import BaseCommand
from attackmate.command import CommandRegistry

@CommandRegistry.register('hello')
class HelloCommand(BaseCommand):
    type: Literal['hello']
    message: str = "Hello, AttackMate!"
```

The `type` field **must** be a unique `Literal` string. It is the discriminator used throughout the codebase to identify this command.

Step 2: Implement the Executor

Create a file in `src/attackmate/executors/` (or a subdirectory). Inherit from `BaseExecutor` and implement `_exec_cmd()`:

```
# src/attackmate/executors/common/helloexecutor.py
from attackmate.executors.baseexecutor import BaseExecutor
from attackmate.result import Result
from attackmate.executors.executor_factory import executor_factory
from attackmate.schemas.hello import HelloCommand

@executor_factory.register_executor('hello')
class HelloExecutor(BaseExecutor):
    async def _exec_cmd(self, command: HelloCommand) -> Result:
        self.logger.info(f"Hello: {command.message}")
        return Result(stdout=command.message, returncode=0)
```

The `_exec_cmd()` method **must** return a `Result` object. `Result` has two fields:

- `stdout`, string output (becomes `$RESULT_STDOUT`)
- `returncode`, integer exit code (becomes `$RESULT_RETURNCODE`)

`BaseExecutor` gives you `self.logger`, `self.varstore`, and all the control-flow mechanics (loops, conditionals, error handling) for free.

Step 3: Add to `__init__.py`

Import your executor in `src/attackmate/executors/__init__.py` and add it to `__all__`:

```
from .common.helloexecutor import HelloExecutor

__all__ = [
    # ... existing entries ...
    'HelloExecutor',
]
```

This ensures the module is imported and the `@executor_factory.register_executor` decorator runs at startup.

Step 4: Add to the `Command` Union

Update `src/attackmate/schemas/loop.py` to include the new command in the `Command` union:

```
from attackmate.schemas.hello import HelloCommand

Command = Union[
    ShellCommand,
    HelloCommand,    # add here
    # ... other commands ...
]
```

Step 5: Add to `RemotelyExecutableCommand`

Update `src/attackmate/schemas/command_subtypes.py`:

```
from attackmate.schemas.hello import HelloCommand

RemotelyExecutableCommand: TypeAlias = Annotated[
    Union[
        HelloCommand,    # add here
        # ... other commands ...
    ],
    Field(discriminator='type'),
]
```

Step 6: (Optional) Add Documentation

Add a `.rst` file in `docs/source/playbook/commands/` and reference it from `docs/source/playbook/commands/index.rst`.

The `BaseExecutor` API

Attribute/Method	What it provides
<code>self.logger</code>	Python logger — use <code>.info()</code> , <code>.warning()</code> , <code>.debug()</code> , <code>.error()</code>
<code>self.varstore</code>	<code>VariableStore</code> — read/write playbook variables
<code>self.varstore.variables</code>	Dict of all current variables
<code>self.varstore.setvar(name, value)</code>	Set a variable programmatically
<code>self.pm</code>	<code>ProcessManager</code> — run subprocesses
<code>self.setoutputvars</code>	Set to <code>False</code> to skip updating <code>\$RESULT_STDOUT</code> etc.

Constructor Arguments

If your executor needs extra configuration (like a connection object or API client), add the argument to your `__init__`, and register it in `_get_executor_config()` in `src/attackmate/attackmate.py`:

```
# In attackmate.py
def _get_executor_config(self) -> dict:
    return {
        # ... existing entries ...
        'my_api_client': self.my_api_client,
    }
```

The `ExecutorFactory` filters constructor kwargs by signature automatically, only kwargs your class actually accepts are passed.

Testing Your Executor

Write a playbook and run it:

```
# test_hello.yml
commands:
  - type: hello
    message: "This is my custom executor working!"

  - type: debug
    cmd: "Executor returned: $RESULT_STDOUT"
```

```
uv run attackmate test_hello.yml
```

You should see your log message and `$RESULT_STDOUT` populated with the return value.